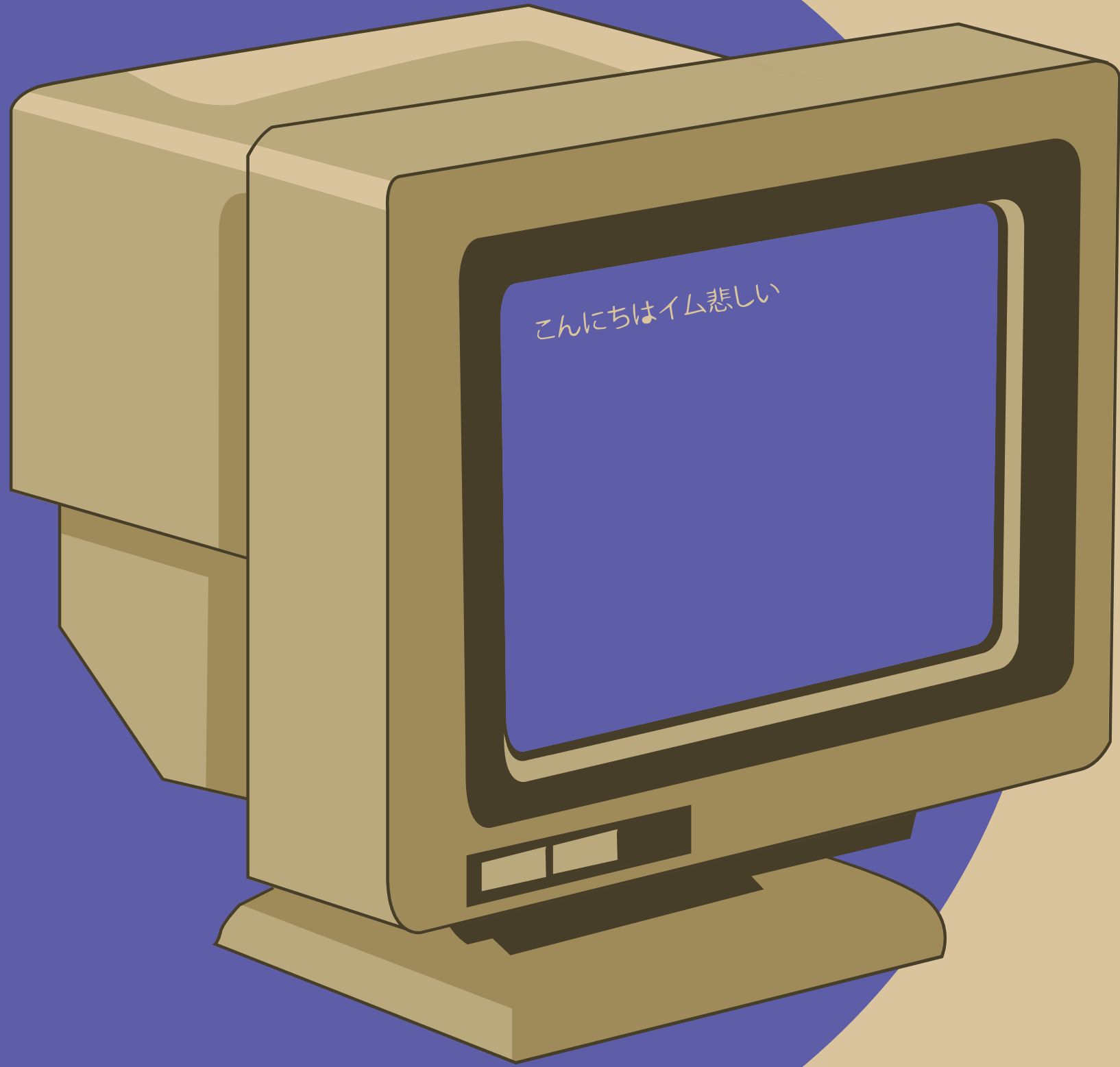
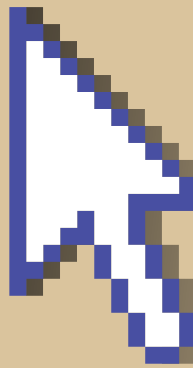




HASHING E SALTING

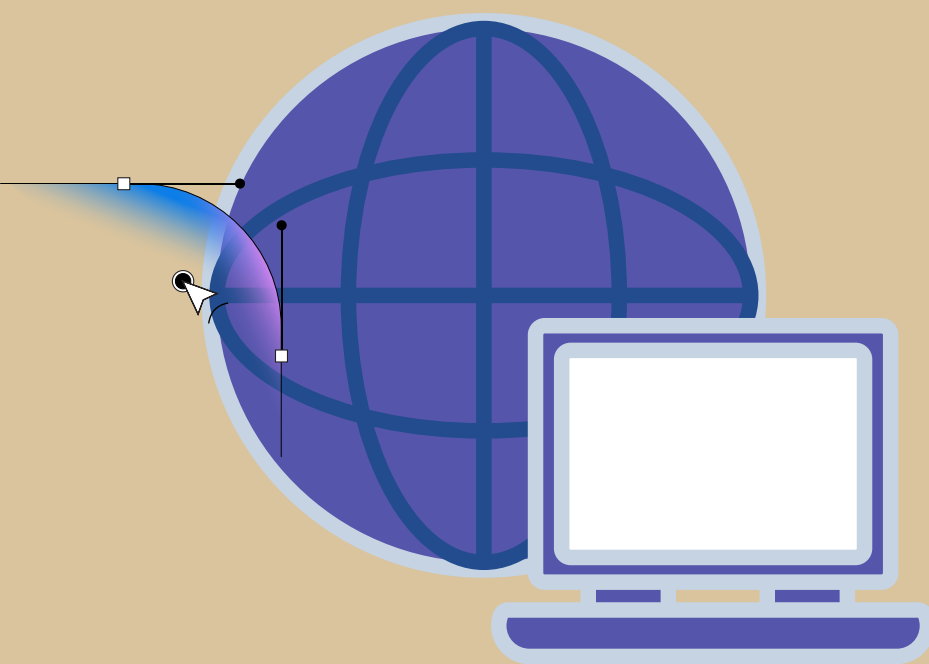




HASHING VS CRIPTOGRAFIA

Criptografia é reversível: Vai e volta. Se você criptografa um dado com uma chave, pode usar a mesma chave para decriptografá-lo.

Hashing é irreversível (Mão única): Uma vez que a senha passa pela função de hash, ela vira uma "sopa de letrinhas" que não pode ser desfeita. Para validar o login, o sistema aplica o hash na senha que o usuário digitou e compara se os hashes batem





PORQUE SALTING

Como o hashing sempre é o mesmo, dada uma string... temos:

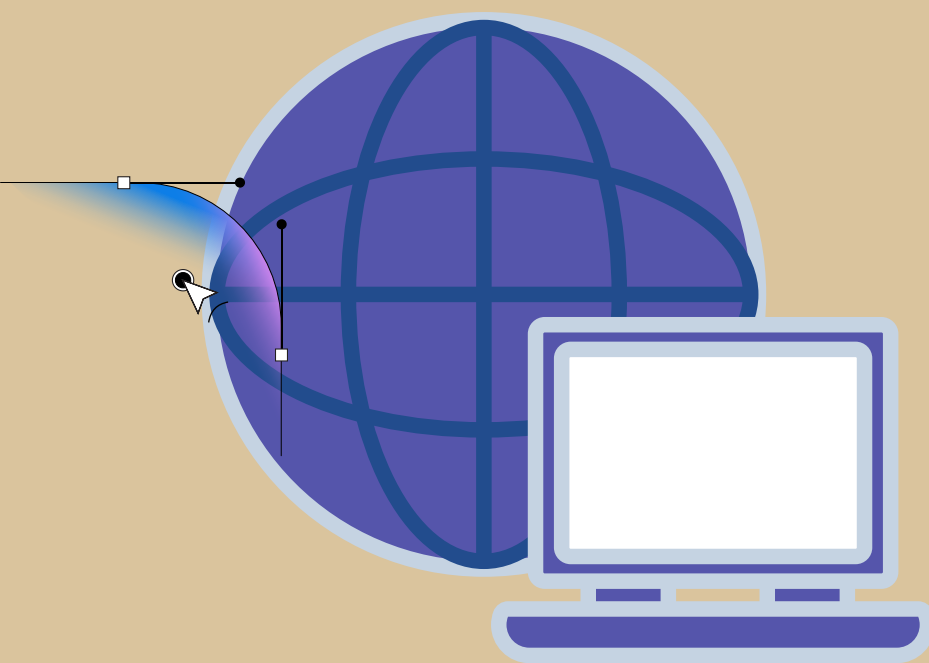
Solução: O Salt (Sal) é uma string aleatória gerada pelo sistema no momento do cadastro e anexada à senha do usuário antes do hashing

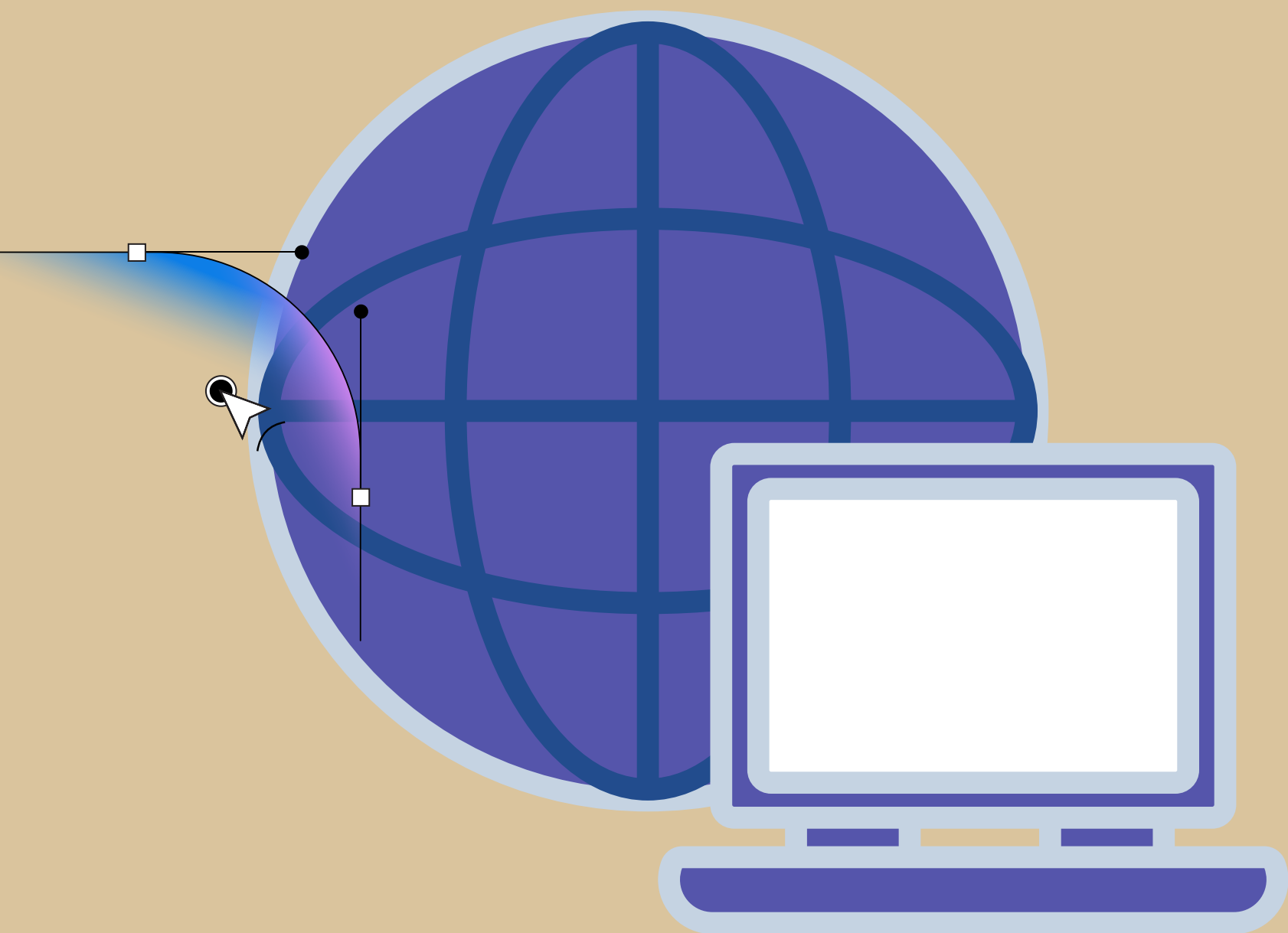
Por exemplo:

João e Maria usarem a mesma senha ("123456"), o hash gerado será idêntico. Um atacante que descubra o hash do João saberá automaticamente a senha da Maria usando "Tabelas Arco-Íris" (Rainbow Tables).

Logo:

$\text{Hash}("123456" + \text{"SaltDoJoao"})$ será completamente diferente de $\text{Hash}("123456" + \text{"SaltDaMaria"})$





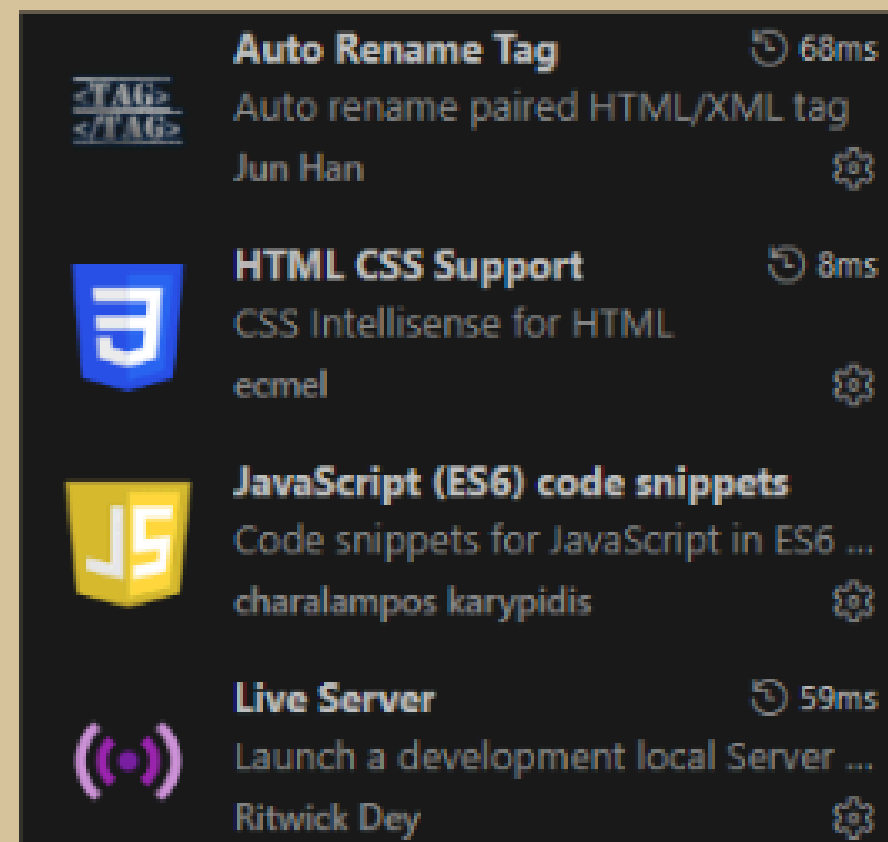
GUIA INSTALAÇÃO

Guia de Instalação: Gerenciador de Notas (Lab de Segurança)

1. Pré-requisitos

Antes de começar, certifique-se de ter instalado:

- **MySQL Workbench.**
<https://dev.mysql.com/downloads/workbench/>
VERSÃO WORKBENCH, geralmente 200mb+
- **VS Code**, com as seguintes extensões



2. Passo a Passo da Montagem

Siga exatamente esta ordem para garantir que o sistema conecte corretamente:

Passo 1: Preparar o Banco de Dados (MySQL)

1. Abra o **MySQL Workbench**.
2. Abra o arquivo scriptGerencia_notas.sql e selecione da linha 1 até a linha 19(ou a ultima onde fecha TABLE notas) e clique no símbolo de raio (as outras linhas são de testes)
3. Isso criará o banco gerenciador_notas e as tabelas usuarios e notas.

Passo 2: Organizar os Arquivos

Crie uma pasta para o projeto e organize os arquivos assim:

```
nomeQualquer/  
├── db.js  
├── server.js  
└── public/  
    ├── index.html
```

Passo 3: Instalar as Dependências

Abra o terminal dentro da pasta do projeto no VS Code e digite:

```
npm init -y  
npm install express mysql2 body-parser cors
```

Passo 4: Configurar a Conexão (db.js)

Abra o arquivo db.js e certifique-se de que o usuário e a senha coincidem com o seu MySQL local:

```
password: 'SUA_SENHA_AQUI',
```

Passo 5: Instalando o Bcrypt (hashing)

```
npm install bcrypt
```

3. Executando o Sistema

No terminal do VS Code, execute o comando:

```
node server.js
```

Se aparecer a mensagem "**Banco de dados MySQL conectado!**" Tá tudo certo.

Como acessar:

Abra o seu navegador e digite:

http://localhost:3000

OU clicar com CTRL + clique abre no VsCode(com a extensão certa)



BCRYPT

VS

ARGON2

- O Bcrypt foi criado em 1999 e se tornou o padrão absoluto da indústria por mais de duas décadas.
- Como ele funciona: Ele exige muito poder de processamento da CPU (processador principal) para calcular o hash, tornando o ataque lento.
- No Mercado: É o mais utilizado no mundo hoje. Praticamente qualquer linguagem (Node.js, Python, PHP, Java) possui suporte nativo ou bibliotecas extremamente estáveis e fáceis de instalar para o Bcrypt. É a escolha segura e padrão para a maioria dos projetos web comuns.

- Ele foi o vencedor oficial do Password Hashing Competition em 2015, sendo eleito o algoritmo mais seguro da atualidade para uso em armazenamento de senhas e derivação de chaves.
- Como funciona: O grande diferencial do Argon2 é que ele não cansa apenas o processador (CPU), ele também exige muita Memória RAM.
- Por que isso importa? Hackers modernos usam placas de vídeo superpotentes (GPUs) ou chips customizados (ASICs) para testar bilhões de senhas por segundo. O Bcrypt sofre um pouco contra esses superchips. O Argon2 destrói essa estratégia, porque esses chips de ataque não têm tanta memória RAM disponível para processar as instruções.
- No Mercado: É considerado o estado da arte e a recomendação máxima para novos sistemas de alta segurança. No entanto, para iniciantes, ele pode ser um pouco mais complexo de instalar em ambientes locais (às vezes exigindo compiladores de C++ na máquina) comparado ao Bcrypt.





HASHING BCrypt COM SALTING

- \$2b\$ \$ -> Indica a versão do algoritmo Bcrypt que foi usada.
- \$10\$ \$ -> É o fator de custo (Cost/Rounds) que você definiu no código (saltRounds = 10). Indica o quão pesado foi o cálculo.
- wI8a5P.wz.yJ6/ -> Estes primeiros 22 caracteres são o Salt aleatório gerado para este usuário.
- uPqZ3.O.R6jO9Y1qZ... -> O restante é o Hash propriamente dito (o resultado matemático da senha misturada com o salt).

